This guide covers how to create custom widgets and components for TamboUI.

Creating Custom Widgets

TamboUI provides two widget interfaces depending on whether your widget needs external state.

Stateless Widgets

For widgets that render the same output given the same inputs:

```
public interface WidgetInterface {
    void render(Rect area, Buffer buffer);
}
```

Example: A custom separator widget:

```
public class Separator implements Widget {
    private final Style style;

    public Separator(Style style) {
        this.style = style;
    }

    @Override
    public void render(Rect area, Buffer buffer) {
        // Fill the entire width with a horizontal line
        for (int x = area.x(); x < area.right(); x++) {
            buffer.set(x, area.y(), new Cell("—", style));
        }
    }
}
```

Stateful Widgets

For widgets that need to track selection, scroll position, or other state:

```
public interface StatefulWidgetInterface<S> {
    void render(Rect area, Buffer buffer, S state);
}
```

Example: A custom counter widget:

```
public class Counter implements StatefulWidget<Counter.State> {
    private final Style style;

    public static class State {
        private int value = 0;

        public int value() { return value; }
        public void increment() { value++; }
        public void decrement() { if (value > 0) value--; }
    }

    public Counter(Style style) {
        this.style = style;
    }

    @Override
    public void render(Rect area, Buffer buffer, State state) {
        String text = "Count: " + state.value();
        int x = area.x();
        for (char c : text.toCharArray()) {
            if (x < area.right()) {
                buffer.set(x++, area.y(), new Cell(String.valueOf(c), style));
            }
        }
    }
}
```

Usage:

```
Counter.State counterState = new Counter.State();
Counter counter = new Counter(Style.EMPTY.bold());
```

```
counter.render(area, buffer, counterState);
```

Buffer Operations

Setting Cells

```
// Set a single cell
buffer.set(x, y, new Cell("X", style));

// Set a string
int col = x;
for (char c : "Hello".toCharArray()) {
    buffer.set(col++, y, new Cell(String.valueOf(c), style));
}
```

Reading Cells

```
Cell cell = buffer.get(x, y);
String symbol = cell.symbol();
Style cellStyle = cell.style();
```

Filling Areas

```
// Fill a rectangle with a character
for (int row = area.y(); row < area.bottom(); row++) {
    for (int col = area.x(); col < area.right(); col++) {
        buffer.set(col, row, new Cell(" ", bgStyle));
    }
}
```

Bounds Checking

Always check bounds before writing:

```
if (x >= area.x() && x < area.right() &&
    y >= area.y() && y < area.bottom()) {
    buffer.set(x, y, cell);
}
```

Exporting Buffer Content

You can export a `Buffer` to SVG, HTML, or plain/ANSI text for screenshots, documentation, or sharing. The API is fluent: start from `export(buffer)` (static import {@code ExportRequest.export}), choose a format, optionally configure options, then write to a file, stream, or string.

Fluent API

```
// Start from a buffer (e.g. after rendering widgets)
Buffer buffer = Buffer.empty(new Rect(0, 0, 80, 24));

// Shorthand: export as SVG with default options
export(buffer).svg().toFile(Path.of("output.svg"));

// Configure options via options(Consumer)
export(buffer).svg()
    .options(o -> o.title("My App"))
    .toFile(Path.of("output.svg"));

// Export to string or bytes
String svgString = export(buffer).svg().toString();
byte[] htmlBytes = export(buffer).html().options(o -> o.inlineStyles(true)).toByte

// Write to an OutputStream or Writer
export(buffer).text().to(outputStream);
```

Format Selection

Use the shorthand methods for the built-in formats:

- `export(buffer).svg()` — SVG (scalable vector graphic)
- `export(buffer).html()` — HTML document (with embedded or external CSS)
- `export(buffer).text()` — Plain text or ANSI-styled text

To use a custom format, pass it to `as(Format)`:

```
export(buffer).as(Formats.SVG).toFile(path);
```

You can also export by file path and let the extension choose the format:

```
export(buffer).toFile(Path.of("screenshot.svg"));   // SVG
export(buffer).toFile(Path.of("page.html"));       // HTML
export(buffer).toFile(Path.of("dump.txt"));        // Plain text
export(buffer).toFile(Path.of("styled.ansi"));     // ANSI-styled text
```

Extension mapping: `.svg` → SVG; `.html` / `.htm` → HTML; `.txt` / `.asc` → plain text; `.ans` / `.ansi` → ANSI text. Unknown extensions default to SVG.

Output Methods

After choosing a format (and optionally configuring options), use:

- `toFile(Path path)` — write to a file (UTF-8)
- `to(OutputStream out)` — write to a stream (UTF-8)
- `to(Writer out)` — write to a writer (caller controls charset; writer is not closed)
- `toString()` — return the exported string
- `toBytes()` — return UTF-8 bytes

Cropping to a region

Use `crop(Rect)` to export only part of the buffer. The rect is clipped to buffer bounds. Returns a new request (immutable); chain with format and output as usual.

```
// Export a single region (e.g. title, table, or footer)
Rect titleRect = new Rect(0, 0, 80, 3);
export(buffer).crop(titleRect).svg().toFile(Path.of("title.svg"));

// Combined rectangle of multiple elements: union of two rects
Rect tableRect = new Rect(22, 3, 58, 12);
Rect footerRect = new Rect(0, 15, 80, 2);
Rect tableAndFooter = tableRect.union(footerRect);
export(buffer).crop(tableAndFooter).svg().toFile(Path.of("table_and_footer.svg"));
```

In Toolkit apps, use `Element.renderedArea()` after a frame is rendered to get an element's last rendered rect, then crop to that rect to export just that element:

```
// After rendering (e.g. in an action handler with access to buffer and element)
Rect area = myPanel.renderedArea();
if (area != null && !area.isEmpty()) {
    export(buffer).crop(area).svg().toFile(Path.of("panel.svg"));
}
```

Default export colors

Export formats use a default foreground and background when a cell style does not specify colors (e.g. after RESET). When you do not pass a resolver, export uses those property defaults.

To supply custom defaults (e.g. from the toolkit style engine), pass a `StylePropertyResolver` via `styles(…)`. The resolver can provide values for the properties `export-foreground` and `export-background`:

```
// Use property defaults (dark theme) — no resolver
export(buffer).svg().toFile(path);

// Custom defaults via a resolver (e.g. from toolkit StyleEngine)
StylePropertyResolver myResolver = StylePropertyResolver.empty();  // e.g. styleEn
export(buffer).svg().options(o -> o.styles(myResolver)).toFile(path);
```

SVG Export

SVG export produces a standalone SVG document suitable for embedding or conversion to PNG/PDF.

Options (`SvgOptions`), via `.options(o → …)` :

- `title(String)` — window/title text (default: `"TamboUI"`)
- `chrome(boolean)` — include window frame, title, and traffic-light buttons (default: `true`); `false` for content only
- `styles(StylePropertyResolver)` — style resolver for default export foreground/background (properties `export-foreground` , `export-background`); when `null` (default), uses `ExportProperties` defaults (dark theme)
- `fontAspectRatio(double)` — character width-to-height ratio for layout (default: 0.61)
- `uniqueId(String)` — prefix for CSS classes and clip paths, or `null` for auto-generated

```
export(buffer).svg()
    .options(o -> o.title("Dashboard"))
    .toFile(Path.of("dashboard.svg"));
```

HTML Export

HTML export produces a full HTML document with styled `<pre>` content. Styles can be embedded in each `<span>` or placed in a `<style>` block.

Options (`HtmlOptions`):

- `styles(StylePropertyResolver)` — style resolver for default export foreground/background (same as SVG); when `null` (default), uses `ExportProperties` defaults
- `inlineStyles(boolean)` — `true` for inline styles on each span (single-file, larger); `false` for a stylesheet (default, smaller HTML)

```
// One file with external stylesheet (default)
export(buffer).html().toFile(Path.of("output.html"));

// Inline styles (self-contained, no separate CSS)
```

```
export(buffer).html()
    .options(o -> o.inlineStyles(true))
    .toFile(Path.of("output_inline.html"));
```

Text Export

Text export produces plain text or ANSI-escape-styled text (e.g. for pasting into terminals or logs).

Options (`TextOptions`):

- `styles(boolean)` — `true` to include ANSI escape codes for colors and modifiers; `false` for plain text only (default)

```
// Plain text
String plain = export(buffer).text().toString();
export(buffer).text().toFile(Path.of("dump.txt"));

// ANSI-styled (e.g. for .ansi files or terminal paste)
String ansi = export(buffer).text().options(o -> o.styles(true)).toString();
export(buffer).text().options(o -> o.styles(true)).toFile(Path.of("styled.ansi"));
```

Creating Toolkit Components

For Toolkit DSL integration, extend the `Component` class. Components use the @OnAction annotation to handle input through the bindings system:

```
public class CounterCard extends Component<CounterCard> {
    private int count = 0;

    @OnAction("increment")
    void onIncrement(Event event) {
        count++;
    }

    @OnAction("decrement")
    void onDecrement(Event event) {
```

```
        count--;
    }

    @Override
    protected Element render() {
        var borderColor = isFocused() ? Color.CYAN : Color.GRAY;

        return panel(() -> column(
                text("Count: " + count).bold(),
                text("Press +/- to change").dim()
        ))
        .rounded()
        .borderColor(borderColor)
        .fill();
    }
}
```

Using Components

Components require an ID:

```
var counter1 = new CounterCard().id("counter-1");
var counter2 = new CounterCard().id("counter-2");

// Define bindings
var bindings = BindingSets.standard()
    .toBuilder()
    .bind(KeyTrigger.ch('+'), "increment")
    .bind(KeyTrigger.ch('='), "increment")
    .bind(KeyTrigger.ch('-'), "decrement")
    .bind(KeyTrigger.ch('_'), "decrement")
    .build();

try (var runner = ToolkitRunner.builder()
        .bindings(bindings)
        .build()) {
    runner.run(() -> row(counter1, counter2));
}
```

CSS Property Support

Widgets can support CSS styling by accepting a `StylePropertyResolver` and using `PropertyDefinition` to define custom properties.

Defining Custom Properties

Define custom properties as static constants using `PropertyDefinition` and register them with `PropertyRegistry`:

```
public class MyGauge implements Widget {
    // Define a custom property for the filled bar color
    public static final PropertyDefinition<Color> BAR_COLOR =
        PropertyDefinition.of("bar-color", ColorConverter.INSTANCE);

    // Register the property so style resolvers recognize it
    static {
        PropertyRegistry.register(BAR_COLOR);
    }

    @Override
    public void render(Rect area, Buffer buffer) {
        // Widget implementation
    }
}
```

> For multiple properties, use `PropertyRegistry.registerAll(PROP1, PROP2, …)`.

Registering properties with `PropertyRegistry` is important because:

- Style resolvers use the registry to validate property names
- Unregistered properties may trigger warnings or errors depending on configuration
- Registration enables the property converter to be used during style resolution

This allows users to style your widget with CSS:

```
MyGauge {
    bar-color: green;
}
```

Using StylePropertyResolver

Widgets accept a `StylePropertyResolver` through their builder to resolve CSS values:

```
public static class MyGaugeWithResolver implements Widget {
    public static final PropertyDefinition<Color> BAR_COLOR =
        PropertyDefinition.of("bar-color", ColorConverter.INSTANCE);

    private final Color barColor;

    private MyGaugeWithResolver(Builder builder) {
        // Resolve: programmatic value → CSS value → property default
        this.barColor = builder.resolveBarColor();
    }

    @Override
    public void render(Rect area, Buffer buffer) {
        // Use barColor for rendering
    }

    public static final class Builder {
        private Color barColor;  // Programmatic value
        private StylePropertyResolver styleResolver = StylePropertyResolver.empty(

        public Builder barColor(Color color) {
            this.barColor = color;
            return this;
        }

        public Builder styleResolver(StylePropertyResolver resolver) {
            this.styleResolver = resolver != null ? resolver : StylePropertyResolv
            return this;
        }

        private Color resolveBarColor() {
            return styleResolver.resolve(BAR_COLOR, barColor);
        }

        public MyGaugeWithResolver build() {
            return new MyGaugeWithResolver(this);
        }
    }
}
```

The resolution order is:

1. **Programmatic value** - value set via builder method (e.g., `.barColor(Color.GREEN)`)
2. **CSS value** - value from the style resolver
3. **Property default** - value from `PropertyDefinition.defaultValue()`

Using Standard Properties

For common properties like `color` and `background` , use `StandardProperties` :

```
// In your resolution method:
Color bg = styleResolver.resolve(StandardProperties.BACKGROUND, background);
Color fg = styleResolver.resolve(StandardProperties.COLOR, foreground);
```

Inheritable vs Non-Inheritable

Inheritance controls whether a property value flows from parent elements to children in the element tree.

Inheritable properties (like `color`) propagate down automatically:

```
Panel {
    color: cyan;  /* All Text elements inside Panel inherit this */
}
```

Non-inheritable properties (like `background`) apply only to the target element:

```
Panel {
    background: gray;  /* Only Panel gets this, not its children */
}
```

When defining your own properties:

- Use `PropertyDefinition.of(…)` for properties that should NOT inherit (most widget-specific properties like `bar-color` , `cursor-color`)

- Use `PropertyDefinition.builder(…).inheritable().build()` for properties that SHOULD inherit (rare - typically only text-related properties)

Most widget-specific properties should be non-inheritable because they control aspects unique to that widget type.

Unicode and Display Width Handling

Terminal display width differs from Java string length. Always use `CharWidth` utilities for width calculations when rendering text.

The Problem

Java's `String.length()` returns the number of UTF-16 code units, not terminal display columns:

Type	Example	`length()`	Display Width
ASCII	"A"	1	1
CJK	"世"	1	2
Simple Emoji	"火"	2	2
ZWJ Emoji	"man-bald"	5	2
Flag Emoji	"flag-GL"	4	2

Using `length()` or `substring()` for display calculations causes:

- ZWJ emoji like "man-bald" to display as "man?" (truncated mid-sequence)
- CJK characters to overflow cells (width 2 treated as width 1)
- Incorrect cursor positioning and alignment

CharWidth Utilities

The `CharWidth` class provides display-width-aware string operations:

```
// Get display width of a string
int width = CharWidth.of("Hello 世界 🔥");  // Returns 14, not 11

// Get display width of a code point
int cpWidth = CharWidth.of(0x4E16);  // Returns 2 (CJK character)

// Truncate to fit display width (preserves grapheme clusters)
String truncated = CharWidth.substringByWidth("Hello 👨‍ World", 8);  // "Hello 👨‍"

// Truncate from the end
String suffix = CharWidth.substringByWidthFromEnd("Hello World", 5);  // "World"

// Truncate with ellipsis
String ellipsized = CharWidth.truncateWithEllipsis(
    "Very long text here",
    10,
    CharWidth.TruncatePosition.END
);  // "Very lo..."
```

Common Patterns

Width Calculation

```
// WRONG - breaks emoji and CJK
int wrongWidth = text.length();

// CORRECT - respects display width
int correctWidth = CharWidth.of(text);
```

Text Truncation

```
// WRONG - may break mid-grapheme cluster
String wrongTruncated = text.substring(0, Math.min(text.length(), maxWidth));
```

```
// CORRECT - preserves grapheme boundaries
String correctTruncated = CharWidth.substringByWidth(text, maxWidth);
```

Position Tracking

```
// WRONG - position drift with wide characters
int col = x;
buffer.setString(col, y, text, style);
col += text.length();

// CORRECT - use display width
int col2 = x;
buffer.setString(col2, y, text, style);
col2 += CharWidth.of(text);
```

Centering Text

```
// WRONG - misaligned with CJK/emoji
int wrongLabelX = x + (width - label.length()) / 2;

// CORRECT - proper centering
int correctLabelX = x + (width - CharWidth.of(label)) / 2;
```

Truncation with Ellipsis

```
// WRONG - char count, not display width
String wrongEllipsis = text;
if (text.length() > maxWidth) {
    wrongEllipsis = text.substring(0, maxWidth - 3) + "...";
}

// CORRECT - using CharWidth utilities
String correctEllipsis = CharWidth.truncateWithEllipsis(text, maxWidth, CharWidth.

// Or manually:
String manualEllipsis = text;
if (CharWidth.of(text) > maxWidth) {
    int ellipsisWidth = CharWidth.of("...");
```

```
    manualEllipsis = CharWidth.substringByWidth(text, maxWidth - ellipsisWidth) +
}
```

Ellipsis Truncation Positions

`CharWidth.truncateWithEllipsis()` supports three positions:

```
String text = "Hello World Example";

// END: "Hello Wor..."
String endTruncate = CharWidth.truncateWithEllipsis(text, 12, CharWidth.TruncatePo

// START: "...d Example"
String startTruncate = CharWidth.truncateWithEllipsis(text, 12, CharWidth.Truncate

// MIDDLE: "Hell...ample"
String middleTruncate = CharWidth.truncateWithEllipsis(text, 12, CharWidth.Truncat
```

You can also use a custom ellipsis string:

```
String customEllipsis = CharWidth.truncateWithEllipsis(text, 12, "…", CharWidth.Tr
```

ZWJ Sequence Safety

Zero-Width Joiner (ZWJ) sequences combine multiple code points into a single visible glyph. `CharWidth.substringByWidth()` automatically preserves these sequences:

```
// "👨‍🦲" is: man (👨) + ZWJ + bald (🦲) = 5 code units, 2 display columns

// Safe truncation - won't break mid-sequence
String result = CharWidth.substringByWidth("A👨‍🦲B", 3);
// Returns "A👨‍🦲" (width 3), not "A👨" (broken sequence showing "?")
```

Reference Implementation

See `Paragraph.java` for a complete example of correct CharWidth usage in a widget that handles text wrapping and truncation.

Best Practices

Widget Design

- Keep widgets focused on a single responsibility
- Use the builder pattern for complex configuration
- Respect the provided `Rect` bounds
- Handle edge cases (empty area, no data)
- Always use `CharWidth` for text width calculations (see Unicode and Display Width Handling)

Performance

- Minimize allocations in `render()` methods
- Pre-compute strings and styles when possible
- Use primitive arrays over collections for large data

State Management

- Keep state classes simple
- Provide methods for all state modifications
- Consider immutable state for thread safety

Native Image Support

When building a native image, you need to make sure that JFR are enabled. This is done by passing `--enable-monitoring=jfr` to the native image builder.

Next Steps

- Review the Widgets Reference to see existing implementations
- Learn about Bindings and Actions for handling input
- Study Application Structure for organizing larger apps

Further Reading

For more details, explore the source code:

- `tamboui-widgets/src/main/java/dev/tamboui/widgets/` - Widget implementations
- `tamboui-toolkit/src/main/java/dev/tamboui/toolkit/` - Toolkit components
- `tamboui-core/src/main/java/dev/tamboui/buffer/` - Buffer and Cell
- `tamboui-core/src/main/java/dev/tamboui/export/` - Export API (SVG, HTML, text)
- `tamboui-core/src/main/java/dev/tamboui/text/CharWidth.java` - Unicode display width utilities
- `tamboui-widgets/src/main/java/dev/tamboui/widgets/paragraph/Paragraph.java` - Reference implementation for CharWidth usage

PREVIOUS
MVC Model

NEXT
API